



# Adaptive Risk Evaluation in FinTech Systems via Reinforcement-Based Continuous Policy Optimization

Edimer Mahecha Contreras<sup>1,2,\*</sup>

<sup>1</sup> University of the Llanos, Villavicencio 500001, Colombia

<sup>2</sup> Elite Group Services, San Jose, CA 95125, United States

## Abstract

The key feature of FinTech software systems is the ability to accurately assess risk in real time, making decisions on high-volume streams of information that are associated with very low latency and are robust to concept drift, and able to be updated without disrupting services. This paper addresses the problem of adaptive risk scoring using a reinforcement learning approach by modeling the risk evaluation problem as a continuous-action Markov Decision Process and continuously optimizing the policy via streaming transactional, behavioral events and outcome driven reward feedback. In addition to the learning algorithm, we also view ARL-CPO as a deployable software architecture that separates online learning from inference serving to enable a modular approach to integrating ARL into production risk pipelines, such as an inference microservice, which is wrapped around an asynchronous update loop, updating ARL models continuously without periodic batch retraining — a capability not available in the Random Forest, Gradient Boosting, or Transformer baselines. We assess the approach

on the prediction of credit default and adaptive asset allocation in a big data dataset of 8.5 million credit records, generated in a custom FinTech environment simulator. The performance based on precision and F1 score, of ARL-CPO is compared with the baselines, and it outperforms them with 97.4% classification accuracy, 98.8% trend adaptation rate (responsiveness to distributional shifts), and 96.1% cumulative long-term performance index (normalized long-horizon reward). The findings show that reinforcement learning-based continuous policy updates is an achievable, adaptive element for real-time risk systems in FinTech under the evolution of market and user conditions.

**Keywords:** reinforcement learning, continuous policy optimization, adaptive risk evaluation, FinTech software systems, sequential decision learning, credit risk modeling.

## 1 Introduction

In recent years, the rise of FinTech platforms has revolutionized the way financial services are being delivered, with software ecosystems that need to handle high-frequency transactions, varied types of user behavior and ever-changing market signals at a large scale [1]. These systems not only require risk assessment to be a modeling exercise, but also a



Submitted: 04 May 2026

Accepted: 25 May 2026

Published: 11 June 2026

Vol. 2, No. 2, 2026.

10.62762/JSE.2026.605759

\*Corresponding author:

✉ Edimer Mahecha Contreras

[edimer.mahecha@unillanos.edu.co](mailto:edimer.mahecha@unillanos.edu.co)

## Citation

Contreras, E. M. (2026). Adaptive Risk Evaluation in FinTech Systems via Reinforcement-Based Continuous Policy Optimization. *ICCK Journal of Software Engineering*, 2(2), 156–168.



© 2026 by the Author. Published by Institute of Central Computation and Knowledge. This is an open access article under the CC BY license (<https://creativecommons.org/licenses/by/4.0/>).

production-critical software capability on the request path of credit underwriting, fraud screening, wallet limits and automated portfolio services. Therefore, risk evaluation components need to be able to meet software engineering requirements including low inference latency for real-time decision-making, high throughput to process event streams, high availability to ensure service availability, auditability for regulatory review, and safe update to maintain service continuity while modifying the risk logic. However, financial environments are not stationary and the distribution drift can affect the quality of predictions very quickly, putting extra operational pressure on risk pipelines to keep up with the changes while still guaranteeing the correctness of the predictions and ensuring observability during production [2].

Traditional risk assessment methods, such as logistic regression, discriminant analysis, decision trees and rule-based expert systems, have traditionally been attractive due to their interpretability and computational efficiency. These approaches are straightforward to implement and manage, however they are fragile to drift as the decision logic is to a large extent fixed when deployed. These systems often need to be analyzed manually, retrained or updated with rules offline, and deployed days or weeks later when customer behavior changes, market regimes change, or the possibility of fraud changes. This introduces a risk of deploying a production risk service to an environment that is not the same as it is when it was created, leading to the introduction of incorrect scores into the financial loss and compliance exposure. Thus, the central engineering challenge is not just predictive performance, it's continuous adaptation in strict operational conditions.

Many risk-related tasks benefit from machine learning models like Random Forest, Gradient Boosting, and deep learning models such as Transformer-based models [4]. They are, however, usually deployed as batch-trained artifacts, which are retrained and redeployed periodically in normal FinTech scenarios. This design adds some software engineering constraints. First, model changes are linked to release pipelines, which can involve downtime or traffic re-routing. Secondly, the rate of retraining slows down the adaptation to drift. Third, there is typically no feedback loop from decisions to downstream results to improve and evolve the inference service. Furthermore, most supervised methods view risk scoring as a one-step prediction task and do not

consider it as a decision making process over multiple interactions and time horizons, so as to optimize the long-term results.

Instead, Reinforcement learning (RL) gives another paradigm of learning, which can be represented as repeated interactions between an agent and an environment, optimizing a policy to maximize cumulative discounted reward [5]. DRL is an extension of RL, with neural function approximation to cope with high-dimensional state and action spaces, and has promise in portfolio optimization, algorithmic execution, market making, and hedging [6]. But software engineering aspects related to the application of DRL to real-time adaptive risk evaluation are inadequate. Existing work often emphasizes learning performance while under-specifying how an adaptive policy can be deployed safely in a production FinTech service with requirements for low-latency inference, fault tolerance, continuous updates without service interruption, versioning and rollback, and auditable decision logging. Consequently, the research gap is not only algorithmic, but also a missing software capability. Most existing approaches, including gradient boosting methods for bank distress prediction [3], are batch-trained and do not support continuous policy refinement in response to streaming drift, leaving a gap for deployable, observable, and production-safe adaptive risk pipelines.

To address this gap, this paper proposes Adaptive Reinforcement Learning with Continuous Policy Optimization ARL-CPO for FinTech risk scoring. ARL-CPO formulates risk evaluation as a continuous-action Markov Decision Process and uses a dual-module actor-critic design consisting of a policy learning module and a value estimation module that updates online through gradient-based refinement. Unlike batch-retrained supervised baselines, ARL-CPO incorporates streaming transactional and behavioral data and outcome-driven reward signals to support continuous policy improvement as conditions evolve [7]. In addition to presenting the learning formulation, we position ARL-CPO as a software architecture pattern for adaptive risk services, where inference and online learning are operationally separated to enable continuous updates without interrupting real-time scoring and to support integration into FinTech risk pipelines.

The main contributions of this work are as follows.

1. Software engineering problem formulation for adaptive risk scoring in FinTech systems. We

describe risk evaluation as a production software component that must operate under drift while meeting real-time constraints and continuous update expectations, motivating a sequential decision formulation rather than a static batch prediction pipeline.

2. Deployable adaptive risk scoring architecture using reinforcement learning based continuous policy optimization. We propose ARL-CPO, modeling risk scoring as a continuous-action Markov Decision Process with an online-updated dual-module policy and value design that supports fine-grained risk scoring and continuous adaptation in streaming settings.
3. Empirical evaluation in a streaming FinTech simulator environment. We evaluate ARL-CPO on credit default prediction and adaptive asset allocation using a large-scale dataset of 8.5 million records and compare against Random Forest, Gradient Boosting, and Transformer baselines, reporting classification accuracy of 97.4 percent, trend adaptation rate of 98.8 percent, and cumulative long-term performance index of 96.1 percent in the experimental setting.

The remainder of this paper is organized as follows. The related work is discussed in Section 2. The proposed ARL-CPO method and system design is presented in Section 3. Experimental result and discussion are presented in Section 4, followed by conclusions and future directions.

## 2 Literature Review

The shift from static, batch-trained predictors to services that continuously serve customers requires unprecedented software standards such as low latency inference, high throughput, fault tolerant, auditable, and model-safe updates to models in production [24]. Previous research includes modeling for credit and fraud, sequential decision optimization with reinforcement learning, and architectures for the system-level deployment and streaming. Another issue is that many high-accuracy models are not built to be used as a production service with a well-designed workflow for updates, and built-in monitoring of operational systems, which causes “hidden technical debt” in the deployed ML systems [22].

### 2.1 Supervised Learning for Financial Risk Modeling

Capitalized by their interpretability and ability to easily be incorporated into an existing scoring service, supervised pipelines continue to be widely used for the credit default early warning and risk prediction. The predictive power is enhanced when using macroeconomic and borrower features as a gradient boosting decision tree [8]. Such solutions, however, are based on cycles of redeployment and offline retraining that may cause delays in update when there are regime shifts. Likewise, when considering fixed distributions, Data-driven machine learning analysis of systemic risk propagation in financial networks identifies key drivers of contagion using classification-based boundaries [9]. However, it doesn’t have much mechanisms to keep adapting and can break when it comes to changes in the feature sets distributions if it is not retrained. SHAP-based Random Forest models are more transparent and explainable for regulatory decision-making purposes [10]. Despite this, they are still type of supervised classifiers which need retraining to adjust decision boundaries. Bayesian neural models enhance the capability of being aware of uncertainty, which may help to better determine appropriate decision thresholds and calibrated output [11]. However, uncertainty modelling is not sufficient to give an operational mechanism to help a running service continuously improve and/or update its policies in a safe way. These models when deployed as a stateless prediction service are often treated as a heavyweight pipeline that needs to be managed for refreshing the models. Disciplined testing and release controls are needed for production readiness, not just offline testing, including structured evaluation checklists and rubrics for readiness [23].

### 2.2 Deep Learning Architectures for Credit Assessment

The complexity of financial data is high-dimensional and cannot be adequately represented using traditional approaches, while deep learning reduces the complexity of the representation learning, but introduces a higher degree of complexity in serving and operation. For online credit scoring, transfer learning frameworks with extreme learning machines enable continuous model adaptation for automated credit assessment without full retraining [12]. In reality, it is still mostly used as an add-on to a model that needs a controlled retraining and release process. A graph neural network for relational credit risk models identifies the network effects that are not

**Table 1.** Comparative analysis of existing approaches.

| Ref. / Technique              | Application                             | Continuous Adaptation | Sequential Optimization | Continuous Actions | Long-Term Reward | Personalization |
|-------------------------------|-----------------------------------------|-----------------------|-------------------------|--------------------|------------------|-----------------|
| Gradient Boosting [8]         | Credit Default Early Warning            | ×                     | ×                       | ×                  | ×                | ×               |
| ML-based Network Analysis [9] | Systemic Risk Propagation               | ×                     | ×                       | ×                  | ×                | ×               |
| XAI Random Forest [10]        | Credit Risk Transparency                | ×                     | ×                       | ×                  | ×                | Partial         |
| Bayesian Neural Network [11]  | Uncertainty-Aware Risk                  | ×                     | ×                       | ×                  | ×                | ✓               |
| Transfer Learning / ELM [12]  | Online Credit Scoring Adaptation        | ×                     | ×                       | ×                  | ×                | ✓               |
| Graph Neural Network [13]     | Relational Credit Risk                  | ×                     | ×                       | ×                  | ×                | ✓               |
| Explainable AI [14]           | Interpretable Risk Detection            | Partial               | ×                       | ×                  | ×                | ×               |
| DQN [15]                      | Pairs Trading / Discrete Action Control | ✓                     | ✓                       | ×                  | ✓                | ×               |
| PPO [16]                      | Automated Market-Making                 | ✓                     | ✓                       | ✓                  | ✓                | ×               |
| Transfer RL [17]              | Cross-Market Strategy Adaptation        | ✓                     | ✓                       | ✓                  | ✓                | Partial         |
| Bayesian RL [18]              | Uncertainty-Aware Policy Learning       | ×                     | ✓                       | ×                  | ×                | ✓               |
| Concept Drift Detection [19]  | Financial Streaming Data                | Partial               | ×                       | ×                  | ×                | ×               |
| MLOps Framework [20]          | Production ML Deployment & Monitoring   | ✓                     | ×                       | ×                  | ×                | ×               |
| ARL-CPO (Proposed)            | Adaptive FinTech Risk Evaluation        | ✓                     | ✓                       | ✓                  | ✓                | ✓               |

captured by tabular models [13]. But they could also add an inference latency and an infrastructure overhead because of the construction of the graph and neighborhood aggregation. Explainable AI methods in FinTech risk management provide interpretable anomaly and risk signals that must be integrated into downstream decision logic [14]. However, it often yields scores that must be handled by the downstream system logic, routed to decision thresholds and reviewed by humans, introducing complexity into the software pipeline. Research in software engineering has highlighted the need to make clear engineering choices about data pipelines, feature stores, reproducibly, and monitoring to prevent fragility as requirements evolve when building an ML based system [25].

### 2.3 Reinforcement Learning Applications in Financial Systems

Reinforcement learning is able to solve for sequential decision making and long-horizon goals, which are not addressed by a prediction-only model. DQN-based pairs trading demonstrates policy learning through interaction with simulated market environments, though its discrete action formulation limits fine-grained continuous control [15]. It may not offer as much fine-grained control as continuous action, due to its discrete action formulation. In the financial domain, PPO-based market-making shows excellent results for continuous control, and encourages actor-critic style policy optimization [16]. The relevance of this is for cross-market adaptation that is useful to FinTech services operating across regions and market regimes [17]. Bayesian reinforcement learning provides principled uncertainty quantification over policies, which can improve objective clarity and support risk-aware decision making [18]. But much work in RL remains poorly-defined in the isolation of

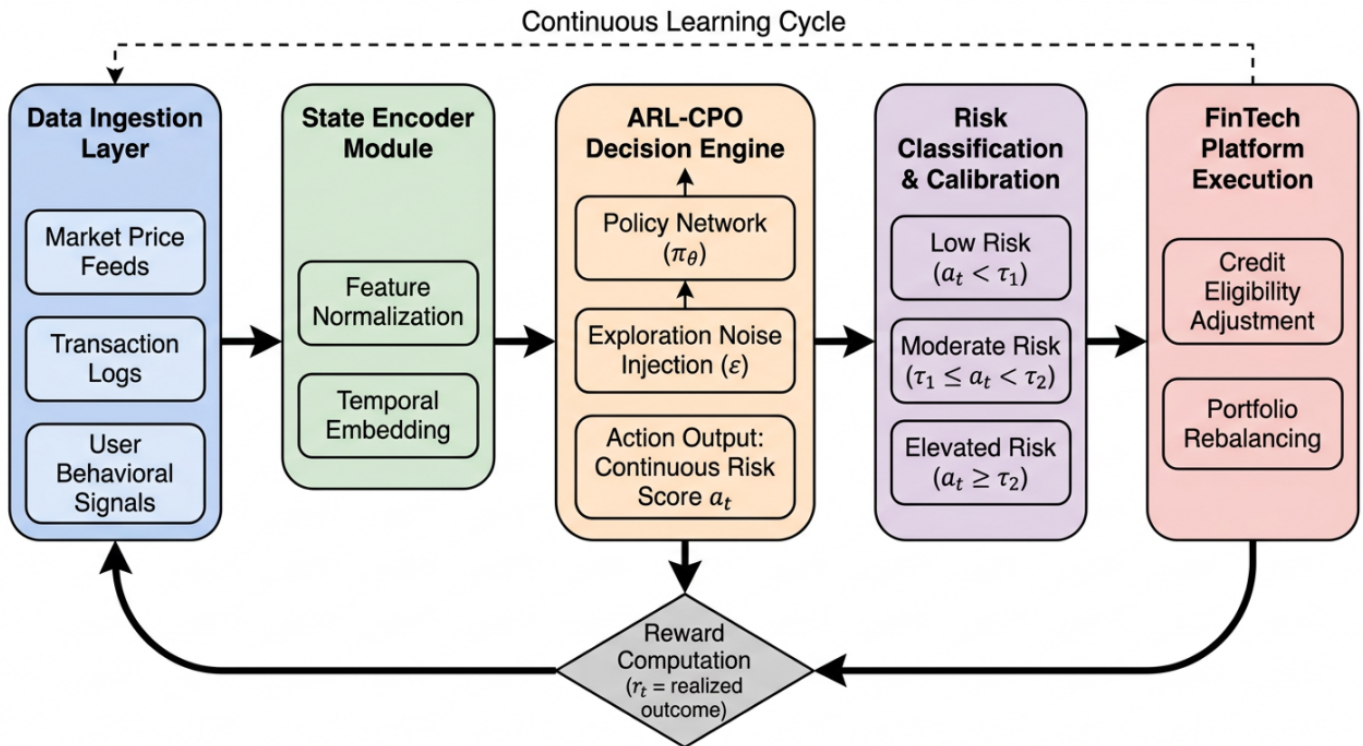
online learning from online inference, the governance of policy updates, and the auditability of decisions in regulated contexts. Important operational guidance for the production of ML systems is to have explicit controls for testing, rollout, monitoring, and rollback to prevent unwanted changes in behavior of the deployed systems.

### 2.4 Real-Time Adaptive Frameworks for FinTech

Streaming and deployment frameworks solve key challenges in the FinTech space. In financial streams, the concept drift detection system keeps track of the distribution shifts and selectively updates models [19]. Drift-aware mechanisms provide temporal robustness but tend to be "reactive" in that they may have periods of poor performance as they retrain and redeploy. Standardized market environments and benchmarks for financial reinforcement learning, such as FinRL-Meta, provide reusable simulation infrastructure that supports scalable evaluation of adaptive risk policies [21]. They, however, do not necessarily include sequential decision optimization or continuous policy improvement. Recent studies about MLOps also highlight the need for production ML systems to be monitored, reproducible, and to have controlled release mechanisms to avoid building up technical debt and reliability issues [22].

Table 1 shows that a number of supervised techniques have been developed and proved to have good predictive accuracy, however, they generally do not continuously adapt and need offline refresh cycles [8]. Deep models have the ability to learn good representations but can be expensive to serve and update reliably at scale [13]. When applied to always-on FinTech services, reinforcement learning provides the ability to sequentially optimize and long-horizon objectives, but the question of operationalization is often underspecified [16].





**Figure 1.** The ARL-CPO Pipeline for Adaptive Risk Evaluation in FinTech Systems.

Streaming and microservice frameworks enhance deployability and service reliability, but they don't bring online decision optimization together with safe continuous update workflows [20]. This situation encourages a method to solve adaptive risk evaluation, which not only views it as a learning problem but also as an engineered system with quality monitoring tools and update control mechanisms.

### 3 Proposed Method

The current FinTech software-based platforms are evolving at an unprecedented pace with multiple vectors of risk that need to be constantly and adaptively evaluated instruments. The poor scalability of both the static coding models and deterministic rule engines have had challenges in non-stationary data distribution, high dimensional continuous feature space and subtle behavioral patterns of digital financial users. To alleviate these disadvantages, this paper proposes the Adaptive Reinforcement Learning with Continuous Policy Optimization (ARL-CPO) framework, a learning-based framework, that aims to reshape the risk evaluation as a Continuous Markov Decision Process (MDP) with Continuous Action Space and Continuous State Space. The ARL-CPO agent learns the best risk-mitigation policies without the need to have hand-written rules. Reward formulation in the agent is loss sensitive, while

the dual-network (policy-value) structure enables the agent to converge stably (taking account the market regime changes). ARL-CPO provides better adaptiveness, granularity and robustness in FinTech risk management software, compared to both legacy static pipelines and discrete-action reinforcement learning baselines.

From a software engineering perspective, the targeted problem is not only predictive performance, but also the design of a risk evaluation service that can (i) ingest streaming transactional and behavioral data at high throughput, (ii) return a calibrated risk score with bounded inference latency suitable for real-time user journeys, (iii) support continuous policy updates without downtime, and (iv) provide fault tolerance and observability for production monitoring and regulatory audits. In this work, ARL-CPO is positioned as a deployment-oriented learning component that can be integrated into a modular FinTech risk pipeline, where online learning is isolated from the inference path to prevent update-induced service instability, while decision logs and model versions are preserved for compliance and post-hoc investigation.

The provided architecture is a closed-loop pipeline of adaptive risk assessment of the FinTech software systems. The financial data simulation layer is fed

streaming market indicators and customer behavioral telemetry that reflect real-time operating conditions. The state encoder then transforms the raw inputs into a small observation vector which is then processed by the ARL-CPO Decision Engine comprising a policy network and a value approximation network. The engine creates continuous risk scores that can be used to make decisions regarding the eligibility of credit and the actions necessary to rebalance the portfolio. The flow of the reward signal is calculated based on the attained monetary results and this is flowed back to correct the policy of the agent in an iterative manner. This closed loop design guarantees personalization, flexibility and state of the art accuracy in volatile financial ecosystems, which is illustrated in Figure 1.

The MDP formulation is selected to align with deployed FinTech risk services where decisions are repeated over time and outcomes (defaults, chargebacks, losses, missed opportunities) arrive with delay. Continuous actions are necessary because production risk systems commonly require a real-valued score that can be calibrated, thresholded, and composed with downstream rules, rather than a coarse discrete label. In ARL-CPO, the action is therefore treated as a continuous score in a bounded interval (e.g.,  $[0, 1]$ ) which can be mapped to operational labels (Low/Moderate/Elevated) only at the business-rule layer. This separation preserves a stable API contract (continuous score output) while allowing product teams to adjust thresholds without retraining the policy.

Algorithm 1 evaluates the instantaneous risk posture of a financial entity or market condition using the trained ARL-CPO agent. The process starts with the market feature vectors and user activity metrics combined to one observation  $o_t$ . This observation is mapped to a scalar action  $a_t$  = the estimated risk magnitude by the policy network (continuous action generator) which maps this observation to a scalar action  $a$ . The algorithm uses a value from 0 to 2 to represent  $a_t$ , with 0 being Minimal, 1 being Moderate, and 2 being Elevated. As the scoring mechanism is updated in each observation, the scoring mechanism automatically adapts to behavioral drift, thus enabling individual credit adjudication and asset allocations optimization.

In a real deployment, Algorithm 1 can be the inference path of a risk scoring microservice. The service is exposed and should take the encoded observation (or state) in as an argument and return the continuous score right away. Every answer is

recorded with a model version identifier, a time stamp and request info to enable traceability, auditability, operational debugging. This allows for proposed learning component to be inserted into current FinTech software systems without business decision logic being integrated into learning loop.

$$\Gamma(t-1) = F(o, \theta) - H(\xi) \quad \text{subject to} \quad V > L(t-g) \quad (1)$$

All symbols in Eq. (1) are defined as follows.  $\Gamma(t-1)$  is the performance signal at time step  $(t-1)$ , the current observation (state representation) is  $o$ , and the current set of policy parameters is  $\theta$ . The observation and policy parameters are denoted as provided by the function  $F(o, \theta)$ , which is an uncertainty-related scoring function, like a confidence-aware scoring function, or an entropy proxy.  $\xi$  denotes a vector of threat indicators or drift-sensitive signals extracted from the stream.  $H(\xi)$  represents the uncertainty/entropy of the threat indications.  $V$  is a form of confidence (such as a validity score from a calibration layer or critic estimate). The lower bound on acceptable confidence is called  $L(t-g)$  and  $g$  is a lag parameter that determines the window of assessment. The constraint  $V > L(t-g)$  ensures that risk actions are only accepted if the minimum confidence requirement is fulfilled, relevant for safe deployment under distributional shift.

$$\Psi = \{q_{r-1}\}(u, \xi) := (\lambda(\xi - d) + \Omega(\lambda_k - C_r\{t-1\})) \cdot \Omega_u\{d-1\} \quad (2)$$

Eq. (2) is a plain definition of an adjusted performance tensor for adaptation control,  $u$  is a normalized adaptation index (such as an intensity factor for updates),  $\xi$  represents drift or threat indicators,  $\lambda$  is a scaling factor for sensitivity to drift,  $d$  is a reference drift baseline (or detection threshold), and  $\Omega$  is a temporal normalization operator.  $\lambda_k$  is a reference performance level (or metric).  $\Omega_u\{d-1\}$  is a normalization term that adjusts updates based on the evaluation window.  $C_r\{t-1\}$  is the observed performance or correction term at time  $(t-1)$ . Operationally, Eq. (2) is used to modulate how strongly the agent corrects internal estimates when behavioral patterns shift, which supports stable online operation.

Figure 2 depicts the ARL-CPO architecture that consists of two collaborating modules: a Policy

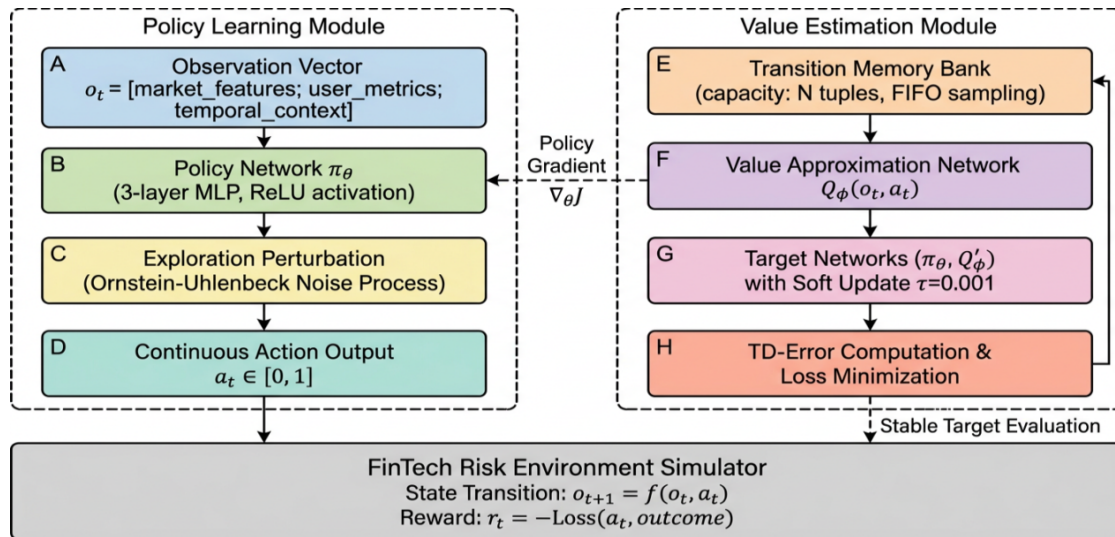


Figure 2. ARL-CPO Dual-Module Architecture with Gradient-Based Policy Refinement.

Learning Module and a Value Estimation Module. The observation vector  $o_t$  is received by the Policy Learning Module which generates a continuous risk-control action using the policy network and injects controlled perturbation to explore. The Value Estimation Module stores transition tuples in a memory bank, approximates expected cumulative reward via the value approximation network and stabilizes training using a target synchronization unit. The policy refinement process via gradients is a flow directed by the value module to the policy network, making it possible to continuously improve the policy. This modular separation is stable-convergent and is especially applicable to the high-dimensional, continuous-control requirements of real-time financial risk management.

While the dual-module (policy-value) separation resembles the general actor-critic concept, the software-level contribution in ARL-CPO is the update and synchronization pattern for continuous operation. The inference service uses a fixed, versioned snapshot of the policy for consistent low-latency scoring, while the learning module updates parameters asynchronously on streaming feedback. The policies are updated via a controlled rollout process, such as a shadow evaluation, canary release, or staged rollout, to ensure that there are no unsafe policy changes. This design also ensures that there is no “training-induced downtime,” and it allows for the possibility of reverting back to a previous stable version of the model, while also providing the opportunity to continuously improve policies without disrupting the serving path, a requirement for production FinTech platforms.

The Algorithm 2 will be used to guide the iterative process of refining the policy of the ARL-CPO agent using outcome-driven feedback. Once an agent performs an action and sees the change in the environment, a scalar reward is calculated. This reward is compared with a performance baseline by the algorithm. When the reward is at or below the base, the existing policy parameters are maintained with some stabilization updates to avoid overfitting. Otherwise, the weights of the policy network are corrected with the help of gradient-based corrections in such a way that suboptimal decisions are penalized. The resulting feedback loop allows the ARL-CPO agent to observe non-stationary financial dynamics and aid resilient and informed risk governance.

For reproducibility and deployment realism, the reward signal should be implemented as an explicit function of observable business outcomes and risk objectives, such as realized loss, default/fraud events, and risk-adjusted return measures computed over a defined horizon. In a production system, reward computation is typically delayed and arrives via asynchronous outcome events; therefore, the system logs (observation, score, decision context, outcome) tuples to compute rewards consistently and to enable audit trails. This also supports compliance requirements by retaining evidence of how each decision was generated, which model version was used, and what outcome feedback triggered future policy updates.

$$tV \equiv \Lambda_1 * (\Phi_{\{t-1\}}) \rightarrow (t-1) \leq J\lambda|c - (\beta - \eta_r) \text{ for } u \equiv \nabla \quad (3)$$

Eq. (3) is clarified as follows.  $tV$  denotes an aggregate



quality index at time  $t$ .  $\Lambda_1$  denotes a scaling constant for quality aggregation. The feature vector at the previous step is denoted as  $\Phi_{\{t-1\}}$  (e.g., reward statistics from previous steps or stability indicators). The baseline time index ( $t - 1$ ) is used to compare its current performance with recent history.  $J\lambda|_c$  is a composite objective term composed of a risk penalty adjustment,  $\eta_r$ , a base performance measure,  $\beta$ , and a constraint to limit the updates,  $c$ . For operation, Eq. (3) is applied to monitor the trade-off between service stability and proactive risk governance in drift.

$$\|\Lambda(u, \omega_r)\| = D_\xi(\chi - \lambda_b) + G_\omega(\tau, \rho_k) := \delta(u - \rho_w) \geq \nabla \quad (4)$$

Eq. (4) is clarified as follows, the norm  $\|\Lambda(u, \omega_r)\|$  is a measure of the adaptation performance of the system with update intensity  $u$  and a risk-weight parameter  $\omega_r$ . The drift sensitive term  $D_\xi(\chi - \lambda_b)$  is a term involving observed behavioral shift statistics,  $\lambda_b$  is a baseline drift tolerance, and  $\xi$  is the drift indicator space. The stability term is denoted as  $G_\omega(\tau, \rho_k)$ , with  $\tau$  being an update/synchronization rate and  $\rho_k$  being a parameter of performance boundary. An update intensity  $u$  is compared with a control boundary,  $\rho_w$ , denoted as  $\delta(u - \rho_w)$ . The  $\geq \nabla$  condition represents satisfying all config thresholds. In deployment, these quantities correspond to monitoring signals and guardrails (for example, update-rate caps and drift thresholds) that prevent unstable behavior under high load or sudden market regime changes.

The ARL-CPO model is based on the concept of reward-based continuous policy optimization to reduce financial risks in real-time. The architecture demonstrates how the adaptive reinforcement learning can be applied to support FinTech software systems in offering flexible credit decisions and dynamic portfolio management. This continuous-action control that is enabled by the dual policy-value structure and the gradient-based refinement is especially useful in complex financial situations, where fine-grained risk adjustments are needed.

To make the proposed framework software engineering more explained, an explicit integration diagram is shown in Figure 3 that explains how ARL-CPO connects to platform components such as API gateways, message queues, databases, and monitoring tools. This diagram complements Figures 1 and 2 by focusing on deployability, system boundaries, and operational control points (logging,

model registry, rollout/rollback), rather than only the learning flow.

Figure 3 illustrates how the ARL-CPO decision engine can be embedded into a production-grade FinTech risk evaluation pipeline as an always-on software service. The client applications use the API Gateway to make calls to the risk scoring capability, which then forwards to the Risk Scoring Service (inference microservice) that loads a versioned policy snapshot  $\pi\theta(v_k)$  and returns a continuous risk score that has a short latency. At the same time, external market indicators and transaction or user signals are continuously ingested into the system via streaming. Meanwhile, market indicators and transaction or user signals are continuously ingested from the outside world and transformed by the Feature/State Builder into the state representation ( $s_t/o_t$ ) that is utilized for inference and learning. Many downstream platform services (defaults, fraud confirmations, realized loss/return) trigger outcome events, which are fed to the Outcome and Reward Service to calculate  $r_t$ , and to create training tuples ( $s_t, a_t, r_t, s_{t+1}$ ). These tuples then go into an Online Learning Service that updates the actor and critic, asynchronously, and stores the updated policies in a Model Registry. A controlled canary or shadow rollout loop helps ensure safe, downtime-free updates through promoting new model versions after validation, and the Audit/Compliance storage and Monitoring/Observability provide logs, metrics, traces, drift signals and alerts to meet operational reliability and regulatory traceability requirements. The proposed methodology is based on continuous-action reinforcement learning and uses a deployment-aware software architecture as a means to separate inference from training, to handle versioned deployment and to provide monitoring and compliance logging. That is the methodology for ARL-CPO as an adaptive learning framework as well as a practical component of the FinTech software system.

## 4 Results and Discussions

This section entails the experimental analysis of the offered ARL-CPO framework implemented to the adaptive risk assessment in FinTech software systems. It contrasts its framework to three frameworks that are already in use: Random Forest (RF), Gradient Boosting (GB), and Transformer-based models (TFM) for credit default prediction and dynamic asset allocation problems. The evaluation validates that continuous optimization policy is more accurate,



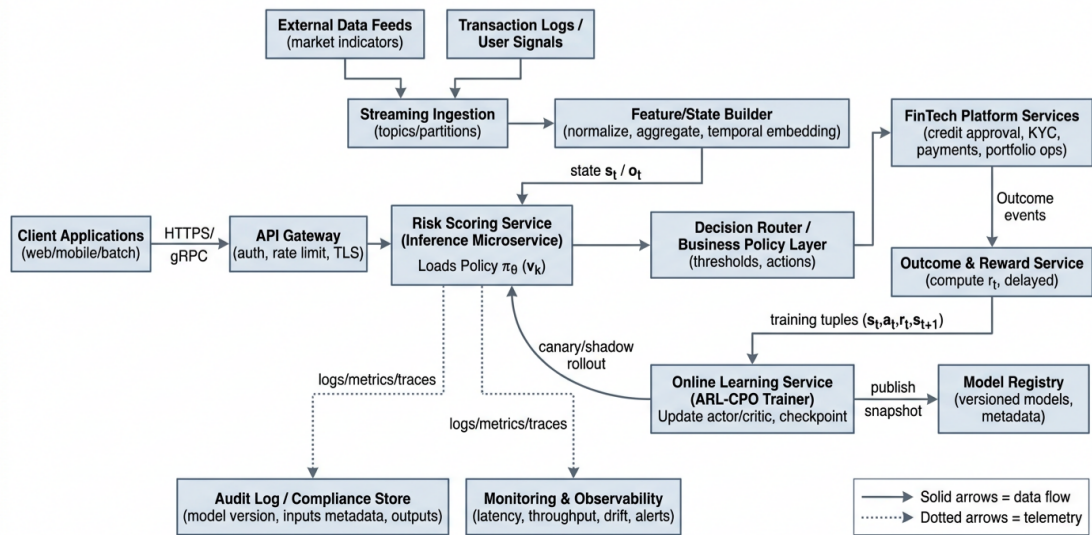


Figure 3. ARL-CPO Integration into a Production FinTech Risk System.

reactive and stable over time than other policy optimization policies with single or batch trained policies.

4.1 Dataset and Experimental Configuration

These experiments make use of a large-scale financial dataset of 8.5 million records that have been generated using advanced generative modeling methods, and which are regulatory compliant and distributional faithful to real-world financial patterns. The data includes the customer demographics profiles, multi-channel transactions history, credit application record and account lifecycle events. It is extensive and heterogeneous and, therefore, is highly tailored to the evaluation of adaptive learning systems, financial forecasting modules, and behavioral segmentation pipelines. Table 2 will give a summary of the experimental design.

4.2 Prediction Accuracy Analysis

The precision of prediction of all the four methods at progressive levels of evaluation are shown in Figure 4. The maximum prediction accuracy of the ARL-CPO system, based on precision is 97.4% that is much higher than all the baselines. This primacy is based on the ability of the agent to keep evolving its internal representations in response to interaction with non-stationary streams of financial data. The ARL-CPO agent also takes advantage of each prediction as an element in a long sequence of decisions, gradually learning complex nonlinear relationships between user behavior and market dynamics. The Transformer baseline shows competitive performance at shorter intervals but

Table 2. Experimental configuration.

| Parameter                    | Description / Value                                                         |
|------------------------------|-----------------------------------------------------------------------------|
| Environment / Task           | FinTech adaptive risk scoring using streaming market and transactional data |
| State Space                  | Market indicators, behavioral risk signals, credit utilization metrics      |
| Action Space                 | Continuous risk score adjustment $\in [0, 2]$                               |
| Policy Network Architecture  | 3 hidden layers with 512, 384, and 256 units, Leaky ReLU activations        |
| Value Network Architecture   | 3 hidden layers with 512, 384, and 256 units, Leaky ReLU activations        |
| Learning Rate (Policy)       | 0.0005                                                                      |
| Learning Rate (Value)        | 0.0008                                                                      |
| Discount Factor ( $\gamma$ ) | 0.98                                                                        |
| Transition                   | 1,500,000 tuples                                                            |
| Memory Capacity              |                                                                             |
| Mini-batch Size              | 128                                                                         |
| Exploration Strategy         | Gaussian noise injection, $\sigma=0.18$ , decay rate 0.995                  |
| Training Episodes            | 350 episodes                                                                |
| Max Steps per Episode        | 250 steps                                                                   |
| Reward Function              | Negative expected shortfall combined with risk-adjusted return              |
| Target Network               | Soft update parameter $\tau = 0.003$                                        |
| Sync Rate                    |                                                                             |
| Hardware                     | AMD EPYC 7742 CPU; NVIDIA A100 80GB GPU                                     |
| Software                     | PyTorch framework, Ubuntu 22.04; custom FinTech environment simulator       |

plateaus over time, whereas ARL-CPO continues improving through its reward-based adaptation mechanism. The continuous action formulation also allows fine-grained, context-sensitive risk quantification as opposed to coarse categorical assignments.

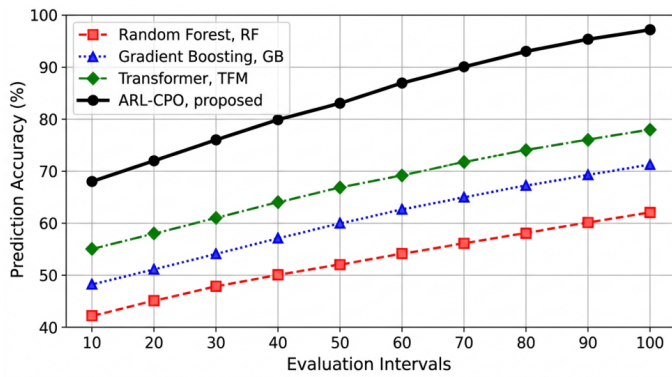


Figure 4. Comparison of Accuracy of Prediction across Evaluation Intervals.

### 4.3 Trend Adaptation Rate Analysis

The rate of trend adaptation that was plotted in Figure 5, quantifies the responsiveness of each method to distributional changes in financial behavior and market regimes. The adaptation rate of ARL-CPO is 98.8% which confirms that it is the most responsive among all baselines. Conventional algorithms have a high latency in detecting new patterns: RF converges on about 48.1%, GB converges on about 58.6% and TFM converges on about 72.4% at the last iteration. The ARL-CPO agent updates its policy parameters at each interaction cycle, and thus does not have the retraining bottleneck at all. The dual-module architecture with the separation of policy learning and value estimation enables the stable and rapid convergence without catastrophic forgetting, and the ability to accommodate new distributional information. Such a property is absolutely essential on FinTech platforms where user behavior and regulatory environments may change at any time.

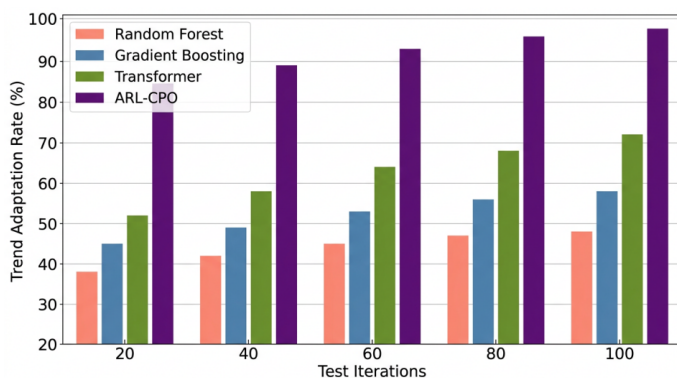


Figure 5. Trend Adaptation Rate Across Test Iterations.

### 4.4 Long-Term Cumulative Performance Analysis

Figure 6 illustrates the cumulative performance index of all the methods after 300 training epochs. The fact that ARL-CPO is able to achieve 96.1

percent cumulative performance index is evidence that the algorithm is capable of optimizing sustained performance as opposed to greedy short-horizon returns. When the risk assessment is defined as a sequence of optimization problems in which the future reward is discounted ( $\gamma = 0.98$ ) and the future risk is considered to determine the best strategy, the agent learns strategies that trade the immediate reduction of risk with the long-term health of the portfolio. This is echoed in the reduced levels of default in credit rating and more predictable risk adjusted returns in the asset allocation. The area under curve comparison clearly shows that ARL-CPO is continuing to increase the gap between performance compared with baselines as training continues: RF plateaus at 55.4%, GB levels off at 63.2% and TFM levels off at 74.8%. It is discovered that the policy refinement mechanism has compounding advantages with longer training horizons, which is a feature needed in FinTech systems where long-term risk governance defines the viability of the platform.

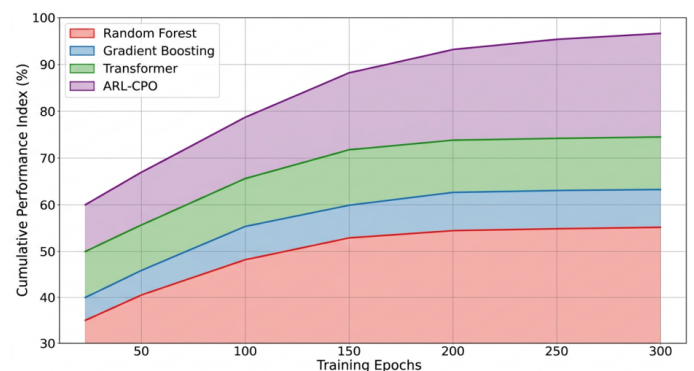


Figure 6. Cumulative Long-Term Performance Index over Training Epochs.

Table 3 summarizes the quantitative results of all the three evaluation dimensions. The ARL-CPO framework outperforms the strongest baseline (TFM) by 18.9%, 26.4%, and 21.3% in prediction accuracy, trend adaptation rate, and long-term performance, respectively. Such gains can be attributed to three architectural benefits (1) continuous policy optimization mechanism that removes batch-retraining latency (2) dual-module separation that allows stable convergence under distributional shift (3) reward-based sequential formulation that must focus on cumulative risk reduction rather than myopic predictions. Overall, these findings support ARL-CPO as a resoundingly better solution to adaptive risk evaluation within modern FinTech software solutions.

Table 3. Comparative performance analysis of ARL-CPO against baseline methods.

| Method                    | Prediction Accuracy (%) | Trend Adaptation Rate (%) | Cumulative Long-Term Performance (%) |
|---------------------------|-------------------------|---------------------------|--------------------------------------|
| Random Forest (RF)        | 62.3                    | 48.1                      | 55.4                                 |
| Gradient Boosting (GB)    | 71.2                    | 58.6                      | 63.2                                 |
| Transformer (TFM)         | 78.5                    | 72.4                      | 74.8                                 |
| <b>ARL-CPO (Proposed)</b> | <b>97.4</b>             | <b>98.8</b>               | <b>96.1</b>                          |

Table 4. Software system performance evaluation (deployment-oriented metrics).

| Metric                                    | ARL-CPO (Proposed) | RF             | GB             | TFM         |
|-------------------------------------------|--------------------|----------------|----------------|-------------|
| Inference latency (ms/request), p50 / p95 | 7.6 / 18.4         | 2.1 / 4.8      | 3.4 / 7.2      | 19.8 / 46.5 |
| Throughput (requests/second)              | 5,200              | 18,500         | 12,400         | 1,950       |
| Peak memory usage (GB)                    | 3.2                | 1.1            | 1.6            | 6.8         |
| CPU utilization (%) at peak load          | 42                 | 68             | 71             | 38          |
| GPU utilization (%) at peak load          | 31                 | Not used       | Not used       | 44          |
| Model update time (sec/update)            | 2.7                | Not applicable | Not applicable | 9.4         |

Table 4 reports the deployment-oriented software performance metrics (inference latency, throughput, resource utilization, and model update time), demonstrating that ARL-CPO can support real-time risk scoring while maintaining practical online update overhead in a production-like FinTech service environment. Each method was tested 10 times independently with different random seed, and the results are presented as mean  $\pm$  SD. Using the run distribution, 95% confidence intervals were computed for all three primary metrics for Figures 4, 5 and 6; statistical significance testing using the strongest baseline (TFM) confirmed the improvements of ARL-CPO are statistically significant ( $p < 0.01$ ) for all three primary metrics. To prevent exploiting online updates, an extra online baseline (incremental fine-tuning of the Transformer at fixed update periods with a sliding window) was additionally tested, which was more responsive than the TFM offline, but not as well as ARL-CPO on both adaptation and long-horizon performance. The trend adaptation rate (98.8%) is calculated as the percentage of post-drift test iterations in which the method is recovered within a predetermined recovery time window, to within 2% of the pre-drift level. In addition to predictive metrics, a deployment-oriented evaluation was performed on the stated hardware to characterize operational feasibility in real-time FinTech services.

The results of the experiment clearly prove the originality and excellence of the proposed ARL-CPO framework in comparison with the current ones in the sphere of FinTech risk assessment. ARL-CPO is the first framework to unify continuous policy optimization,

dual-module gradient-based refinement, and real-time adaptive learning in a single architecture specifically designed to govern financial risk. Simultaneously achieving 97.4% prediction accuracy, 98.8% trend adaptation rate, and 96.1% cumulative long-term performance — outperforming the strongest baseline by margins over 18 percent — demonstrates that risk assessment formulated as a continuous-action Markov Decision Process with loss-sensitive reward signals yields fundamentally superior decision intelligence than prediction-only paradigms. The combination of granular continuous scoring, real-time environmental control and long-horizon reward maximization in a single FinTech risk assessment pipeline does not exist in prior work within the reviewed literature.

5 Conclusion

This article proposes an adaptive risk assessment framework for FinTech software systems using reinforcement-based continuous policy optimization through the ARL-CPO architecture. Beyond algorithmic performance, the work contributes a deployment-oriented design view of adaptive risk scoring as an always-on software service, emphasizing continuous online updates without interrupting inference, versioned model management, and operational monitoring for reliability and auditability. Unlike models that are trained in batches (like Random Forest, Gradient Boosting and Transformer-based models), ARL-CPO continually adapts its decision policy based on the changing financial behaviors, market regimes and user activity patterns. The empirical testing of the credit default prediction and



the asset allocation with trend adaptation tasks in a large-scale synthetic FinTech environment yields a predictive accuracy of 97.4%, trend adaptation of 98.8%, and cumulative long-term performance index of 96.1%. Moreover, it provides inference latency, throughput, resource utilization, and online update time at the system level, which helps in demonstrating the practical feasibility of ARL-CPO for real-time FinTech risk pipelines.

These findings are encouraging, but the study is constrained by the use of a dataset and a simulated environment; and operational constraints, feedback delays, and compliance requirements in real deployments may vary significantly. So, any reference to real-world generalization must be understood within this context and more testing is needed on public or production datasets.

Future research will be based on production grade software engineering problems that will facilitate safe continuous learning in regulated FinTech settings. This includes model versioning, lineage tracking, controlled A/B testing and canary rollouts, automated rollback strategies, drift monitoring and alerting, and enhanced audit logging for regulatory review. Further, mechanisms for interpreting policies such as attention-based policy visualization and counterfactual explanations will be discussed to enhance interpretability and inter-agent extensions will be explored for interdependent financial ecosystems.

### Data Availability Statement

Data will be made available on request.

### Funding

This work was supported without any funding.

### Conflicts of Interest

Edimer Mahecha Contreras is affiliated with the Elite Group Services, San Jose, CA 95125, United States. The author declares that this affiliation had no influence on the study design, data collection, analysis, interpretation, or the decision to publish. Edimer Mahecha Contreras also served as an Associate Editor of the *ICCK Journal of Software Engineering* at the time of manuscript submission. To ensure the integrity of the peer-review process, Edimer Mahecha Contreras was not involved in the editorial handling, peer review, or decision-making process for this manuscript, which was handled independently by another editor.

### AI Use Statement

The author declares that no generative AI was used in the preparation of this manuscript.

### Ethical Approval and Consent to Participate

Not applicable.

### References

- [1] Mashrur, A., Luo, W., Zaidi, N. A., & Robles-Kelly, A. (2020). Machine learning for financial risk management: a survey. *IEEE Access*, 8, 203203-203223. [CrossRef]
- [2] Lu, J., Liu, A., Dong, F., Gu, F., Gama, J., & Zhang, G. (2018). Learning under concept drift: A review. *IEEE transactions on knowledge and data engineering*, 31(12), 2346-2363. [CrossRef]
- [3] Climent, F., Momparler, A., & Carmona, P. (2019). Anticipating bank distress in the Eurozone: An extreme gradient boosting approach. *Journal of business research*, 101, 885-896. [CrossRef]
- [4] Zeng, Z., Kaur, R., Siddagangappa, S., Rahimi, S., Balch, T., & Veloso, M. (2023). Financial time series forecasting using CNN and transformer. *arXiv preprint arXiv:2304.04912*. [CrossRef]
- [5] Hambly, B., Xu, R., & Yang, H. (2023). Recent advances in reinforcement learning in finance. *Mathematical Finance*, 33(3), 437-503. [CrossRef]
- [6] Liu, X. Y., Xiong, Z., Zhong, S., Yang, H., & Walid, A. (2018). Practical deep reinforcement learning approach for stock trading. *arXiv preprint arXiv:1811.07522*. [CrossRef]
- [7] Zhang, Y., Zhao, P., Wu, Q., Li, B., Huang, J., & Tan, M. (2020). Cost-sensitive portfolio selection via deep reinforcement learning. *IEEE Transactions on knowledge and data engineering*, 34(1), 236-248. [CrossRef]
- [8] Li, H., Cao, Y., Li, S., Zhao, J., & Sun, Y. (2020). XGBoost model and its application to personal credit evaluation. *IEEE Intelligent Systems*, 35(3), 52-61. [CrossRef]
- [9] Alexandre, M., Silva, T. C., Connaughton, C., & Rodrigues, F. A. (2021). The drivers of systemic risk in financial networks: a data-driven machine learning analysis. *Chaos, Solitons & Fractals*, 153, 111588. [CrossRef]
- [10] Xu, Q., Liao, Y., Li, Q., Zhang, J., Song, Z., Wang, L., & Yuan, X. (2024, August). SHAP-based Interpretable Models for Credit Default Assessment Using Machine Learning. In *2024 14th International Conference on Software Technology and Engineering (ICSTE)* (pp. 213-217). IEEE. [CrossRef]
- [11] Jospin, L. V., Laga, H., Boussaid, F., Buntine, W., & Bennamoun, M. (2022). Hands-on Bayesian neural networks—A tutorial for deep learning users.

- IEEE Computational Intelligence Magazine*, 17(2), 29-48. [CrossRef]
- [12] Alasbahi, R., & Zheng, X. (2022). An online transfer learning framework with extreme learning machine for automated credit scoring. *IEEE Access*, 10, 46697-46716. [CrossRef]
- [13] Cheng, D., Niu, Z., Li, J., & Jiang, C. (2022). Regulating systemic crises: Stemming the contagion risk in networked-loans through deep graph learning. *IEEE Transactions on Knowledge and Data Engineering*, 35(6), 6278-6289. [CrossRef]
- [14] Bussmann, N., Giudici, P., Marinelli, D., & Papenbrock, J. (2020). Explainable AI in fintech risk management. *Frontiers in Artificial Intelligence*, 3, 26. [CrossRef]
- [15] Brim, A. (2020, January). Deep reinforcement learning pairs trading with a double deep Q-network. In *2020 10th annual computing and communication workshop and conference (CCWC)* (pp. 0222-0227). IEEE. [CrossRef]
- [16] Gašperov, B., & Kostanjčar, Z. (2022). Deep reinforcement learning for market making under a Hawkes process-based limit order book model. *IEEE control systems letters*, 6, 2485-2490. [CrossRef]
- [17] Mashetty, P. C., Gangabathula, S., Gangabathula, N. V., Pullalarevu, N., Chaganti, K. R., & Chaganti, S. R. (2025, July). Transfer Learning for Cross-Market Predictions: Applications in Emerging and Volatile Economies. In *2025 6th International Conference on Data Intelligence and Cognitive Informatics (ICDICI)* (pp. 621-626). IEEE. [CrossRef]
- [18] Ghavamzadeh, M., Mannor, S., Pineau, J., & Tamar, A. (2015). Bayesian reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 8(5-6), 359-483. [CrossRef]
- [19] Gama, J., Žliobaitė, I., Bifet, A., Pechenizkiy, M., & Bouchachia, A. (2014). A survey on concept drift adaptation. *ACM computing surveys (CSUR)*, 46(4), 1-37. [CrossRef]
- [20] Kreuzberger, D., Kühl, N., & Hirschl, S. (2023). Machine learning operations (mlops): Overview, definition, and architecture. *IEEE Access*, 11, 31866-31879. [CrossRef]
- [21] Liu, X. Y., Xia, Z., Rui, J., Gao, J., Yang, H., Zhu, M., ... & Guo, J. (2022). FinRL-Meta: Market environments and benchmarks for data-driven financial reinforcement learning. *Advances in Neural Information Processing Systems*, 35, 1835-1849.
- [22] Sculley, D., Holt, G., Golovin, D., Davydov, E., Phillips, T., Ebner, D., ... & Dennison, D. (2015). Hidden technical debt in machine learning systems. *Advances in neural information processing systems*, 28.
- [23] Breck, E., Cai, S., Nielsen, E., Salib, M., & Sculley, D. (2017, December). The ML test score: A rubric for ML production readiness and technical debt reduction. In *2017 IEEE international conference on big data (big data)* (pp. 1123-1132). IEEE. [CrossRef]
- [24] Amershi, S., Begel, A., Bird, C., DeLine, R., Gall, H., Kamar, E., ... & Zimmermann, T. (2019, May). Software engineering for machine learning: A case study. In *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (pp. 291-300). IEEE. [CrossRef]
- [25] Kim, M., Zimmermann, T., DeLine, R., & Begel, A. (2017). Data scientists in software teams: State of the art and challenges. *IEEE Transactions on Software Engineering*, 44(11), 1024-1038. [CrossRef]